CSE 31
Computer Organization

Lecture 3 – C Pointers

Announcements

- Lab
  - Lab 1 due week after next (with 7 days grace period* after due date)
    - » Demo is REQUIRED to receive full credit**
  - Lab 2 out on week 4
    - » Due at 11:59pm on the same day of your lab during week 5 (with 7 days grace period* after due date)
    - » You must demo your submission to your TA within 14 days from posting of lab
    - » Demo is REQUIRED to receive full credit**
- Reading assignment
  - Chapter 4-6 of K&R (C book) to review C/C++ programming

* There is a 10% penalty for late submissions during the grace period

** Submissions without demo will only receive partial credit for the TPS Activity portion of the assignment

Quiz:

```
void main(); {
  int *p, x=5, y; // init
  y = *(p = &x) + 1;
  int z;
  flip-sign(p);
  printf("x=%d, y=%d, *p=%d\n", x, y, p);
}
flip-sign(int *n){*n = -(*n)}
```

How many syntax+logic errors in this C99 code?

#Errors
a) 1
b) 2
c) 3
d) 4
e) 5

Quiz: Answer

```
void main();{
   int *p, x=5, y; // init
   y = *(p = &x) + 1;
   int z;
   flip-sign(p);
   printf("x=%d, y=%d, *p=%d\n", x, y, *p);
}
flip-sign(int *n){*n = -(*n);}
```

How many syntax+logic errors in this C99 code?

5…

(signed ptr print is logical err)

#Errors
a) 1
b) 2
c) 3
d) 4
e) 5

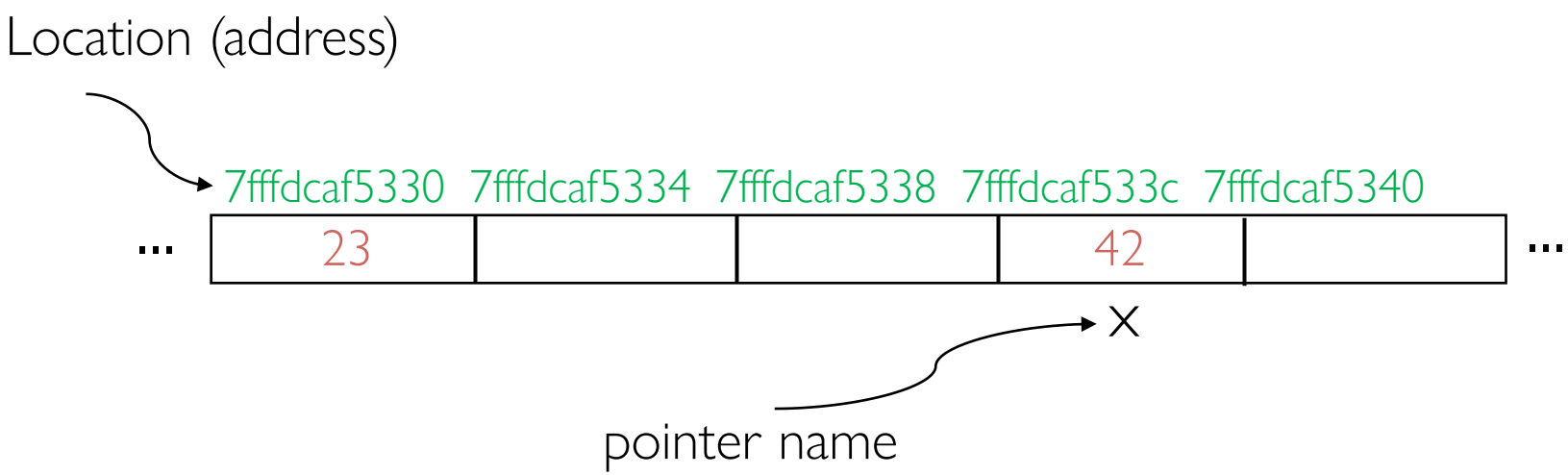
C Pointers

Address vs. Value

- Consider memory to be a single huge array:
  - Each cell of the array has an address associated with it.
  - Each cell also stores some value.
  - Do you think addresses use signed or unsigned numbers?
    - » Negative address?!
- Don't confuse the address referring to a memory location with the value stored in that location.

Pointers

- An address refers to a particular memory location. In other words, it points to a memory location.
- Pointer: A variable that contains the address of a variable.



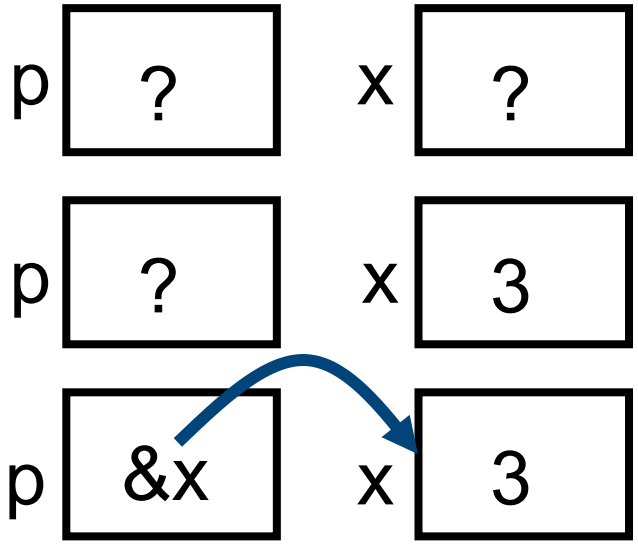
Pointers

- How to create a pointer:
  & operator: get address of a variable

```
int *p, x;

x = 3;

p = &x;
```



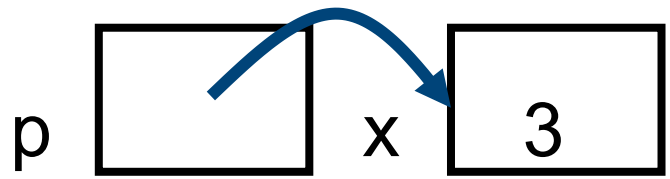
Note the "*" gets used 2 different ways in this example. In the declaration to indicate that p is going to be a pointer, and in the printf to get the value pointed to by p.

- How to get a value pointed to?
  - * "dereference operator": get value pointed to

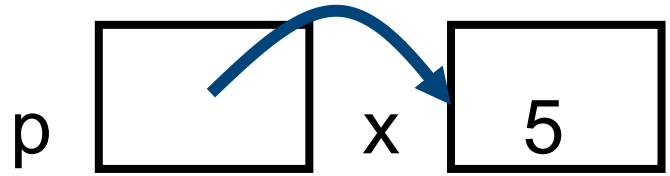
```
printf("p points to %d\n",*p);
```

Pointers

- How to change a variable pointed to?
  - Use dereference * operator on left of =




```
*p = 5;
```



Pointers and Parameter Passing

- C (and C++) pass parameters "by value"
  - procedure/function/method gets a copy of the parameter, so changing the copy cannot change the original

```
void addOne (int x) {
    x = x + 1;
}

int main() {
    int y = 3;
    addOne(y);
    return 0;
}
```

What is `y` after calling `addOne(y)`?

Pointers and Parameter Passing

- How to get a function to change a value?

```
void addOne (int *p) {
    *p = *p + 1;
 }
…
int y = 3;
addOne(&y);
…
```

`int *p = &y` when addOne() is called.

Passing the reference of y

`y` is now = 4 after the program ends.

Pointers

- Pointers are used to point to any data type (`int`, `char`, a `struct`, etc.).

- Normally a pointer can only point to one type (`int`, `char`, a `struct`, etc.).
  - `void *` is a type that can point to anything (generic pointer)
  - Use sparingly to help avoid program bugs… and security issues… and a lot of other bad things!

Pointers & Allocation (1/2)

- After declaring a pointer:

```
int *ptr;
```

  `ptr` doesn't actually point to anything yet (it actually points somewhere - but we don't know where!).

- We can either:
  - make it point to something that already exists, or
  - allocate room in memory for something new that it will point to… (we will talk about it later)

C Pointer Dangers

- C lets you cast a value of any type to any other type <u>without</u> performing any checking.

```
int x = 1000;
int *p = x;          /* invalid?? */
int *q = (int *) x; /* valid */
```

- The first pointer declaration should be invalid since the types do not match. (unsigned vs signed)
  - Does C allow this?

- The second declaration is valid in C but is almost certainly wrong
  - Is it ever correct?

More C Pointer Dangers

- Declaring a pointer just allocates space to hold the pointer – it does not *necessarily* allocate anything to be pointed to!
- Local variables in C are not initialized, they may contain anything.
- What does the following code do?

```
void f() {
    int *ptr;
    *ptr = 5;
}
```

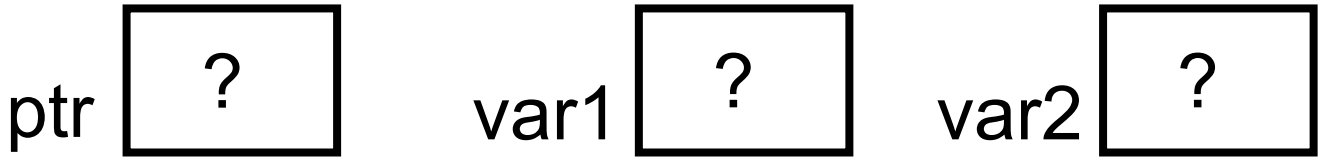
Where does it store the "5"?

Pointers & Allocation (2/2)

- Pointing to something that already exists:

```
int *ptr, var1, var2;
```

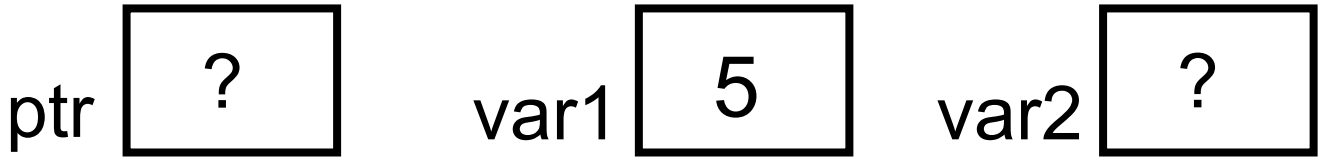
- `var1` and `var2` have room implicitly allocated for them.



Pointers & Allocation (2/2)

- Pointing to something that already exists:

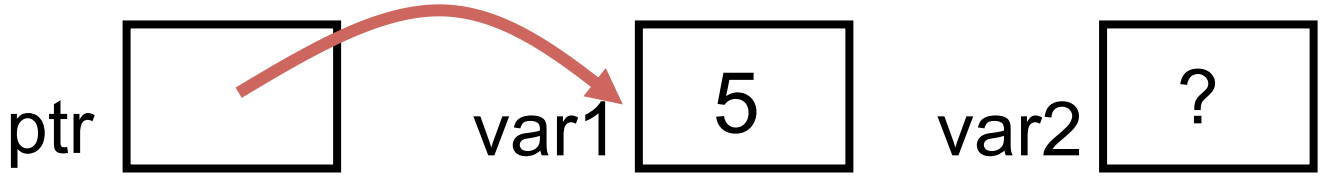
```
int *ptr, var1, var2;
var1 = 5;
```



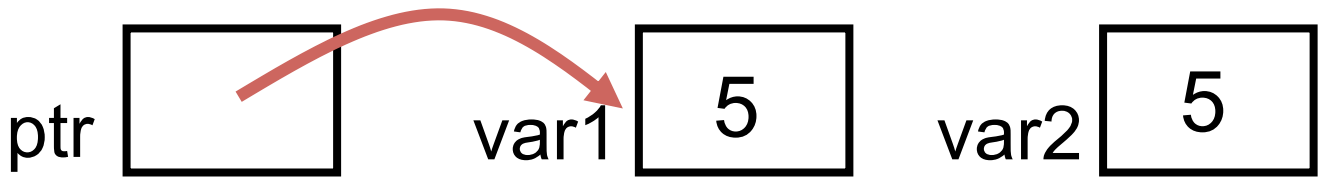
Pointers & Allocation (2/2)

- Pointing to something that already exists:

```
int *ptr, var1, var2;
var1 = 5;
ptr  = &var1;
```



Pointers & Allocation (2/2)

- Pointing to something that already exists:

```
int *ptr, var1, var2;
var1 = 5;
ptr  = &var1;
var2 = *ptr;
```